

Instituto Tecnológico de las Américas (ITLA)

Nombre: Miguel Angel Peña Acevedo

Matricula: 2025-0741

Materia: Seguridad de redes

Carrera: Seguridad informática

Tema: Networking

Maestro: Jonathan Esteban Rondon Corniel

GitHub: <https://github.com/Itsmiguelps/Ataques-y-defensas-de-redes/tree/main>

Video: <https://www.youtube.com/playlist?list=PL7fyKcoaUM3-yGXB4a5rSAdiru8yLqHr9>



ATAQUES DOCUMENTADOS

01	CDP DoS	Denegación de Servicio vía Cisco Discovery Protocol
02	ARP MitM	Man-in-the-Middle mediante Envenenamiento ARP
03	DHCP Spoofing	Servidor DHCP Falso (Rogue DHCP)
04	DHCP Starvation	Agotamiento del Pool DHCP
05	MAC Flooding	Desbordamiento de la Tabla CAM del Switch
06	STP Root Claim	Usurpación del Root Bridge en Spanning Tree

Objetivo del Laboratorio

El presente laboratorio tiene como objetivo demostrar, en un entorno controlado con PNetLab y Kali Linux Docker, seis ataques que explotan vulnerabilidades en protocolos de Capa 2 del modelo OSI. Cada ataque se ejecuta mediante un script Python con Scapy, y se documenta su funcionamiento, sus parámetros, el proceso de ejecución y la contramedida correspondiente para mitigarlo.

Objetivos específicos

1. Comprender cómo los protocolos de Capa 2 (CDP, ARP, DHCP, STP, Ethernet) pueden ser explotados al carecer de mecanismos de autenticación nativos.
2. Implementar y ejecutar los seis ataques en el entorno de laboratorio PNetLab con Kali Docker.
3. Verificar el impacto de cada ataque en los dispositivos Cisco de la topología.
4. Aplicar y validar las contramedidas de mitigación sobre switches y Router Cisco IOS.
5. Documentar evidencia del ataque y su solución en formato profesional.

Información del Entorno

Asignatura	Seguridad en Redes
Entorno	PNetLab + Kali Linux Docker + Cisco IOSvL2
Herramienta	Python 3 + Scapy (python3-scapy)
Ataques	6 ataques de Capa 2 del modelo OSI
Contramedidas	Una por cada ataque — configuración Cisco IOS

Documentación de la Red — Topología

El laboratorio está hecho en PNetLab con nodos Kali Linux Docker y switches/Router Cisco IOSvL2. Cada ataque utiliza una topología mínima y independiente con los nodos necesarios para realizar los ataques.

Tabla de Direccionamiento General

Nodo	Interfaz	Dirección IP / Rol	Función
Kali Linux	eth1	DHCP	Acceso internet (instalar paquetes)
Kali Linux	eth2	7.41.10.50/24	Interfaz de ataque (red de Lab)
Kali Linux	eth2	7.41.1.50/24	Interfaz de ataque (ARP MitM)
PC-Víctima 1	eth1	7.41.1.10/24	Víctima en ataques MitM
PC-Víctima 2	eth1	7.41.1.20/24	Segunda víctima en ARP MitM
Router R1	e0/0	7.41.10.1/24	Gateway + servidor DHCP
DHCP Pool	—	.101 – .254	154 IP disponibles (léase 2 h)
SW1	—	VTP Server	Root Bridge · Priority 4096
SW2, SW3	—	VTP Client	Priority 4096

VLAN Configuradas

VLAN ID	Nombre	Uso en el Laboratorio
10	ADMIN	Red de administración — ataques STP, CDP, DHCP, MAC Flooding
20	USERS	Red de usuarios — ataques ARP MitM

Topologías por Ataque

CDP DoS y ARP MitM

Nodos: SW1 (switch core con trunk), SW2 (cliente), Kali (eth2), PC-Víctima 1, PC-Víctima 2.

Interfaz / Nodo	Rol y Dirección IP
SW1 — e0/0	Trunk VLAN 10, 20 hacia SW2
SW2 — e1/0	Trunk VLAN 10, 20 (hacia SW1)
SW2 — e1/1	Trunk VLAN 10, 20 (hacia Kali / víctimas)
Kali — eth2	7.41.1.50/24 — Atacante
PC-Víctima 1 — eth1	7.41.1.10/24
PC-Víctima 2 — eth1	7.41.1.20/24

DHCP Spoofing y DHCP Starvation

Nodos: Router R1 (servidor DHCP + gateway), Kali (eth2), PC-Víctima.

Interfaz / Nodo	Rol y Dirección IP
R1 — e0/0	7.41.10.1/24 — Gateway + DHCP Server
DHCP Pool	7.41.10.101 – 7.41.10.254 (excluye .1–.100)
Kali Atacante — eth2	7.41.10.50/24
PC-Víctima — eth1	DHCP dinámico (obtiene IP del pool legítimo o malicioso)

MAC Flooding

Nodos: Switch Cisco IOSvL2, Kali (eth2), PC-Víctima. Sin router requerido.

Interfaz / Nodo	Rol
Kali — eth2	7.41.10.50/24 — Atacante
PC-Víctima	7.41.10.100/24
Switch — e0/0	Puerto de acceso hacia Kali
Objetivo	Tabla CAM del switch (8192 entradas)

STP Root Claim

Nodos: SW1 (Root Bridge legítimo, priority 4096), SW2 y SW3 (VTP Client), Kali. Topología con 3 switches para demostrar la reconvergencia STP.

Interfaz / Nodo	Configuración
SW1	VTP Server · PVST priority 4096 · Root Bridge legítimo
SW1 — e0/0	Access VLAN 10 (hacia Kali)
SW1 — e1/0	Trunk VLAN 10 (hacia SW2)
SW1 — e0/3	Trunk VLAN 10 (hacia SW3)
SW2 — e0/0	Trunk VLAN 10 (hacia SW1)
SW2 — e0/1	Trunk VLAN 10 (hacia SW3)
SW3 — e0/0	Trunk VLAN 10 (hacia SW1)
SW3 — e0/1	Trunk VLAN 10 (hacia SW2)
Kali — eth2	7.41.10.50/24 — conectado a SW1 e0/0
VLAN	VLAN 10 ADMIN · VTP domain: EMPRESA

Resumen de Ataques y Contramedidas

La siguiente tabla presenta una vista consolidada de todos los ataques documentados, sus protocolos, impacto y contramedidas:

#	Ataque	Protocolo	Impacto	Contramedida
01	CDP DoS	CDP — Cisco L2	Agota tabla CDP, CPU/RAM alta	no cdp run / no cdp enable
02	ARP MitM	ARP — RFC 826	Intercepta todo el tráfico (MitM)	ip arp inspection + port-security max 1

03	DHCP Spoofing	DHCP — RFC 2131	MitM automático + DNS malicioso	ip dhcp snooping + trust en uplink
04	DHCP Starvation	DHCP — RFC 2131	DoS: pool agotado, clientes sin IP	port-security max 2 + violation shutdown
05	MAC Flooding	Ethernet 802.3	Switch en modo hub → sniffing total	port-security + mac-address sticky
06	STP Root Claim	STP 802.1D/PVST+	Control de topología, DoS 30–50 s	bpduguard enable + guard root

Instalación de dependencias

```

-----
para que el .py funcione script
apt update && apt install python3-scapy -y
-----

acceso remoto para pegar los script
apt update && apt install openssh-server -y
service ssh start
-----

```

Configuración de red

```

pc atacante-----
-----

ip del dispositivo para la lan atacante

ip addr flush dev eth2
ip addr add 7.41.10.50/24 dev eth2
ip link set eth2 up
ip route add default via 7.41.10.1

para tener internet
ip link set eth1 up

dhclient eth1

```

```

-----
ip forwarding
sudo sysctl -w net.ipv4.ip_forward=1

echo 1 > /proc/sys/net/ipv4/ip_forward

```

Funcionamiento, Parámetros y Contramedidas

CDP DoS. Denegación de Servicio vía Cisco Discovery Protocol Protocolo: CDP

Objetivo del Laboratorio

Demostrar cómo el protocolo CDP (Cisco Discovery Protocol) puede ser explotado para causar una denegación de servicio en switches Cisco, agotando su tabla de vecinos y saturando los recursos de CPU y memoria del dispositivo objetivo.

Inundando la tabla de vecinos CDP de un switch Cisco con cientos de entradas falsas mediante paquetes CDP contruidos con Scapy, causando agotamiento de CPU y memoria del dispositivo objetivo.

Objetivo del Script

Inundar la tabla de vecinos CDP de un switch Cisco con cientos de entradas falsas construidas con Scapy, causando agotamiento de CPU y memoria. Los paquetes son enviados a la dirección multicast CDP 01:00:0C:CC:CC:CC con TLV aleatorios.

Parámetros del Script

Parámetro	Descripción	Default
-i / --interface	Interfaz de red de salida	eth2
-c / --count	Número de paquetes CDP a enviar	500
-d / --delay	Retardo en segundos entre paquetes	0.005 s
-v / --verbose	Mostrar detalle de cada paquete enviado	Desactivado

Requisitos para Utilizar la Herramienta

- Python 3 con Scapy: apt install python3-scapy -y
- Privilegios root (ejecutar con sudo)
- CDP habilitado en el switch objetivo
- Kali en el mismo dominio L2 que el switch

Funcionamiento del Script

El script construye paquetes CDP válidos estructuralmente usando: Ethernet → LLC (DSAP/SSAP=0xAA) → SNAP (OUI=0x00000C, code=0x2000) → CDP Header → TLV. Cada paquete contiene TLV con datos aleatorios: Device ID (ROUTER-XXXXXX), Software Versión, Platform y PortID. Los paquetes se envían a la dirección multicast CDP 01:00:0C:CC:CC:CC. El switch intenta aprender cada vecino nuevo, agotando la tabla CDP y elevando el uso de CPU.

Ejecución del Ataque

```
ejecución del ataque-----  
|  
ejecucion basica  
sudo python3 cdp_dos.py -i eth2 -c 1000 -d 0.001
```

Verificación del Ataque

verificación-----

show processes cpu history

show cdp neighbors

```
Switch>en
Switch#show cdp neighbors
Capability Codes: R - Router, T - Trans Bridge, B - Source Route Bridge
                  S - Switch, H - Host, I - IGMP, r - Repeater, P - Phone,
                  D - Remote, C - CVTA, M - Two-port Mac Relay

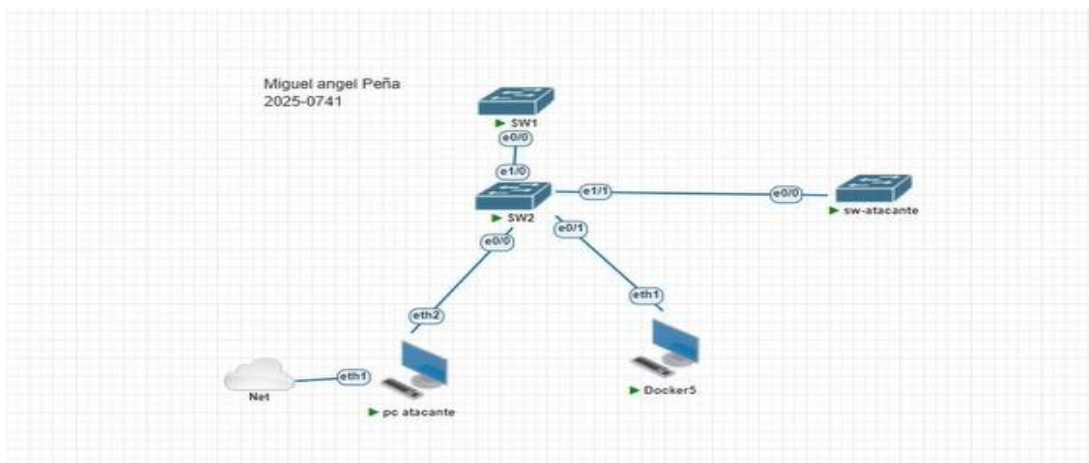
Device ID         Local Intrfce   Holdtme    Capability   Platform   Port ID
Switch            Eth 1/1        131        R S I       Linux Uni  Eth 0/0
Switch            Eth 1/0        161        R S I       Linux Uni  Eth 0/0

Total cdp entries displayed : 2
Switch#show cdp neighbors
Capability Codes: R - Router, T - Trans Bridge, B - Source Route Bridge
                  S - Switch, H - Host, I - IGMP, r - Repeater, P - Phone,
                  D - Remote, C - CVTA, M - Two-port Mac Relay

Device ID         Local Intrfce   Holdtme    Capability   Platform   Port ID
ROUTER-U4SIE8     Eth 0/0        161        R S I       4500       Gig 0/13
ROUTER-LV09PD     Eth 0/0        160        R S I       2960       Gig 0/11
ROUTER-9K4LFH     Eth 0/0        168        R S I       4500       Gig 0/44
ROUTER-U100WH     Eth 0/0        168        R S I       4500       Gig 0/4
ROUTER-K0318E     Eth 0/0        164        R S I       4500       Gig 0/12
ROUTER-XS47WZ     Eth 0/0        179        R S I       3750       Gig 0/6
ROUTER-IA1RX1     Eth 0/0        173        R S I       2960       Gig 0/5
ROUTER-X1W7SX     Eth 0/0        170        R S I       7200       Gig 0/3
ROUTER-K6N6JL     Eth 0/0        171        R S I       3750       Gig 0/40
ROUTER-FDVT65     Eth 0/0        170        R S I       2960       Gig 0/16
ROUTER-GPH1Z7     Eth 0/0        164        R S I       4500       Gig 0/40
ROUTER-XX0SMR     Eth 0/0        163        R S I       2960       Gig 0/12
ROUTER-LJ9I0F     Eth 0/0        172        R S I       4500       Gig 0/27
ROUTER-SAY08C     Eth 0/0        170        R S I       3750       Gig 0/48
ROUTER-KCFDPZ     Eth 0/0        168        R S I       3750       Gig 0/31
ROUTER-NIKIJO     Eth 0/0        160        R S I       3750       Gig 0/16
ROUTER-QQAEDA     Eth 0/0        174        R S I       2960       Gig 0/2
ROUTER-F4PZ0I     Eth 0/0        173        R S I       2960       Gig 0/43
ROUTER-UZ8CL9     Eth 0/0        163        R S I       7200       Gig 0/10
--More--
```

Topologia

Nodos: SW1 + SW2 (trunks VLAN 10,20) + Kali (eth2) conectada a SW2. CDP habilitado globalmente en ambos switches.



Salida del script

```
Documents Music Templates Thinstall drives
admin@Docker:~$ sudo python3 cdp_dos.py -i eth2 -c 1000 -d 0.001
python3: can't open file '/home/admin/cdp_dos.py': [Errno 2] No such file or directory
admin@Docker:~$ sudo python3 cdp_prueba.py -i eth2 -c 1000 -d 0.001
No protocol specified
No protocol specified

#####
666 CDP DoS FLOOD ATTACK - LABORATORIO DE SEGURIDAD 666
666 SOLO PARA USO EDUCATIVO Y AUTORIZADO! 666
#####

[*] Iniciando CDP DoS Flood en interfaz: eth2
[*] Paquetes a enviar: 1000 | Delay: 0.001s

[-] Progreso: 1/1000 paquetes | 13.7 pkt/s
[-] Progreso: 51/1000 paquetes | 16.1 pkt/s
[-] Progreso: 101/1000 paquetes | 16.3 pkt/s
[-] Progreso: 151/1000 paquetes | 16.9 pkt/s
[-] Progreso: 201/1000 paquetes | 16.9 pkt/s
[-] Progreso: 251/1000 paquetes | 16.8 pkt/s
[-] Progreso: 301/1000 paquetes | 16.7 pkt/s
[-] Progreso: 351/1000 paquetes | 16.6 pkt/s
[-] Progreso: 401/1000 paquetes | 16.7 pkt/s
[-] Progreso: 451/1000 paquetes | 16.7 pkt/s
[-] Progreso: 501/1000 paquetes | 16.7 pkt/s
[-] Progreso: 551/1000 paquetes | 16.8 pkt/s
[-] Progreso: 601/1000 paquetes | 16.7 pkt/s
[-] Progreso: 651/1000 paquetes | 16.7 pkt/s
[-] Progreso: 701/1000 paquetes | 16.7 pkt/s
[-] Progreso: 751/1000 paquetes | 16.7 pkt/s
[-] Progreso: 801/1000 paquetes | 16.7 pkt/s
[-] Progreso: 851/1000 paquetes | 16.7 pkt/s
[-] Progreso: 901/1000 paquetes | 16.7 pkt/s
[-] Progreso: 951/1000 paquetes | 16.7 pkt/s
```

Verificación del daño

```
Capability Codes: R - Router, T - Trans Bridge, B - Source Route Bridge
S - Switch, H - Host, I - IGMP, r - Repeater, P - Phone,
D - Remote, C - CVTA, M - Two-port Mac Relay

Device ID      Local Intrfce  Holdtime    Capability    Platform    Port ID
ROUTER-FZQ46I  Eth 0/0       149         7200          Gig 0/21
ROUTER-3CV7ER  Eth 0/0       134         3750          Gig 0/5
ROUTER-U4SIE8  Eth 0/0       113         4500          Gig 0/13
ROUTER-N87R0E  Eth 0/0       145         7200          Gig 0/21
ROUTER-ST30MN  Eth 0/0       142         2960          Gig 0/12
ROUTER-QIW21K  Eth 0/0       133         3750          Gig 0/19
ROUTER-LV09PD  Eth 0/0       112         2960          Gig 0/11
ROUTER-3PBL3Z  Eth 0/0       160         7200          Gig 0/41
ROUTER-66Z84R  Eth 0/0       158         7200          Gig 0/17
ROUTER-V02W0H  Eth 0/0       157         4500          Gig 0/44
ROUTER-7D1DDF  Eth 0/0       146         7200          Gig 0/40
ROUTER-01NRXA  Eth 0/0       142         7200          Gig 0/45
ROUTER-897310  Eth 0/0       141         3750          Gig 0/7
ROUTER-62K80C  Eth 0/0       140         2960          Gig 0/47
ROUTER-3WNSIR  Eth 0/0       133         2960          Gig 0/19
ROUTER-9K4LFH  Eth 0/0       120         4500          Gig 0/44
ROUTER-U100WH  Eth 0/0       120         4500          Gig 0/4
ROUTER-K0318E  Eth 0/0       117         4500          Gig 0/12
ROUTER-3G45I9  Eth 0/0       152         3750          Gig 0/13
ROUTER-VVZVYP  Eth 0/0       145         3750          Gig 0/13
ROUTER-NA3D02  Eth 0/0       144         3750          Gig 0/36
ROUTER-3M6LCB  Eth 0/0       136         7200          Gig 0/17
ROUTER-PWCM95  Eth 0/0       131         3750          Gig 0/11
ROUTER-XS47WZ  Eth 0/0       129         3750          Gig 0/6
ROUTER-3807YW  Eth 0/0       161         3750          Gig 0/17
ROUTER-81BPTM  Eth 0/0       160         3750          Gig 0/6
ROUTER-JU0XMG  Eth 0/0       157         4500          Gig 0/21
ROUTER-GZ8ES1  Eth 0/0       138         2960          Gig 0/32
ROUTER-ECWHQJ  Eth 0/0       136         7200          Gig 0/47
ROUTER-IA1RX1  Eth 0/0       123         2960          Gig 0/5
ROUTER-X1W7SX  Eth 0/0       120         7200          Gig 0/3
ROUTER-NYR0E3  Eth 0/0       163         7200          Gig 0/14
ROUTER-P9LSL9  Eth 0/0       148         4500          Gig 0/24
ROUTER-CCEK49  Eth 0/0       146         7200          Gig 0/42
ROUTER-L1K20V  Eth 0/0       143         2960          Gig 0/20
ROUTER-1JV2DX  Eth 0/0       134         3750          Gig 0/8
ROUTER-K6N6JL  Eth 0/0       121         3750          Gig 0/40
ROUTER-FDVT65  Eth 0/0       120         2960          Gig 0/16
ROUTER-GPH1Z7  Eth 0/0       114         4500          Gig 0/40
ROUTER-XX0SMR  Eth 0/0       113         2960          Gig 0/12
ROUTER-HUKEAR  Eth 0/0       155         4500          Gig 0/44
ROUTER-1BSLX4  Eth 0/0       155         3750          Gig 0/16
ROUTER-4L7EMG  Eth 0/0       151         7200          Gig 0/1
ROUTER-NFGREX  Eth 0/0       142         4500          Gig 0/17
ROUTER-LJ9I0F  Eth 0/0       122         4500          Gig 0/27
```


Contra medida y verificación

Deshabilitar CDP globalmente o por interfaz. Se recomienda deshabilitarlo en todos los puertos de acceso hacia usuarios finales.

```
Device ID      Local Intrfce  Holdtme  Capability  Platform  Port ID
Switch         Eth 1/1        167      R S I       Linux Uni  Eth 0/0

Total cdp entries displayed : 1
Switch#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
Switch(config)#interface e0/0
Switch(config-if)#no cdp enable
Switch(config-if)#exit
Switch(config)#exit
Switch#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
Switch(config)#
*Jun  3 02:52:43.723: %SYS-5-CONFIG_I: Configured from console by console
Switch(config)#exit
Switch#show cdp neighbors
*Jun  3 02:52:46.679: %SYS-5-CONFIG_I: Configured from console by console
Switch#show cdp neighbors
Capability Codes: R - Router, T - Trans Bridge, B - Source Route Bridge
                  S - Switch, H - Host, I - IGMP, r - Repeater, P - Phone,
                  D - Remote, C - CVTA, M - Two-port Mac Relay

Device ID      Local Intrfce  Holdtme  Capability  Platform  Port ID
Switch         Eth 1/0        153      R S I       Linux Uni  Eth 0/0
Switch         Eth 1/1        131      R S I       Linux Uni  Eth 0/0

Total cdp entries displayed : 2
Switch#show cdp neighbors
Capability Codes: R - Router, T - Trans Bridge, B - Source Route Bridge
                  S - Switch, H - Host, I - IGMP, r - Repeater, P - Phone,
                  D - Remote, C - CVTA, M - Two-port Mac Relay

Device ID      Local Intrfce  Holdtme  Capability  Platform  Port ID
Switch         Eth 1/0        128      R S I       Linux Uni  Eth 0/0
Switch         Eth 1/1        160      R S I       Linux Uni  Eth 0/0

Total cdp entries displayed : 2
Switch#show cdp neighbors
Capability Codes: R - Router, T - Trans Bridge, B - Source Route Bridge
                  S - Switch, H - Host, I - IGMP, r - Repeater, P - Phone,
                  D - Remote, C - CVTA, M - Two-port Mac Relay

Device ID      Local Intrfce  Holdtme  Capability  Platform  Port ID
Switch         Eth 1/0        126      R S I       Linux Uni  Eth 0/0
Switch         Eth 1/1        158      R S I       Linux Uni  Eth 0/0
```

ARP MitM Man-in-the-Middle mediante Envenenamiento ARP

Protocolo: ARP (RFC 826)

Objetivo del Laboratorio

Demostrar cómo el protocolo ARP puede ser explotado para interceptar el tráfico entre dos hosts, posicionando a Kali como intermediario transparente (Man-in-the-Middle) sin que ninguna de las víctimas lo detecte.

Objetivo del Script

Interceptar todo el tráfico entre dos víctimas mediante el envenenamiento de sus tablas ARP. Con IP forwarding activo, Kali actúa como intermediario transparente. Al detener el ataque, el script restaura las tablas ARP originales.

Parámetros del Script

Parámetro	Descripción	Default
-t1	IP de la víctima 1	7.41.1.10
-t2	IP de la víctima 2	7.41.1.20
-i / --iface	Interfaz de red del atacante	eth2

Requisitos para Utilizar la Herramienta

- Python 3 con Scapy: apt install python3-scapy -y
- Privilegios root (sudo)
- IP forwarding habilitado: echo 1 > /proc/sys/net/ipv4/ip_forward
- Kali en el mismo segmento L2 que las víctimas

Funcionamiento del Script

La script habilita IP forwarding para reenviar el tráfico interceptado de forma transparente.

Envía periódicamente (cada 1 s) ARP Replies no solicitados a ambas víctimas:

```
-> Le dice a Victima 1 que la IP de Victima 2 tiene la MAC de Kali  
-> Le dice a Victima 2 que la IP de Victima 1 tiene la MAC de Kali
```

Ambas víctimas actualizan su caché ARP y enrutan el tráfico a través de Kali (MitM).

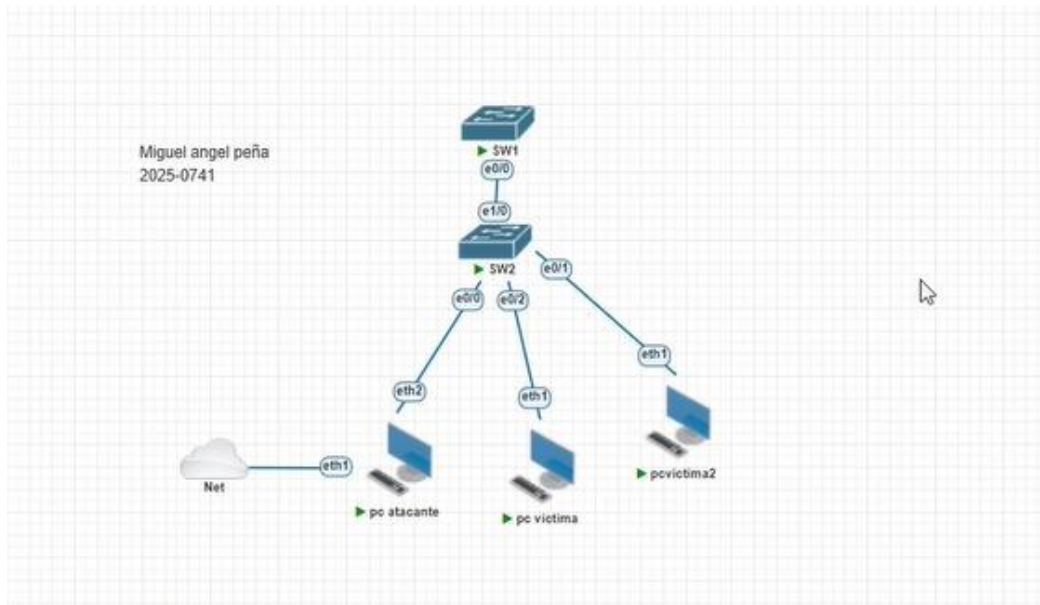
Al detener con Ctrl+C, el script restaura las tablas ARP originales con las MAC reales.

Ejecución del Ataque

```
ejecución del ataque  
sudo python3 arp_mitm.py -t1 7.41.1.10 -t2 7.41.1.20 -i eth2  
  
# Habilitar IP forwarding (obligatorio)  
echo 1 > /proc/sys/net/ipv4/ip_forward  
|
```

Topología

Nodos: SW1 + SW2 (trunks VLAN 10,20) + Kali (eth2=7.41.1.50) + PC-Victima 1 (.1.10) + PC-Victima 2 (.1.20).



Salida del script

```
python3: can't open file '/home/admin/arp_mitm.py': [Errno 2] No such file or directory
root@pc atacante:/home/admin# sudo python3 mitm.py -t1 7.41.1.10 -t2 7.41.1.20 -i eth2
sudo: unable to resolve host pc atacante: Name or service not known
No protocol specified
No protocol specified
[*] Envenenando: 7.41.1.10 <-> 7.41.1.20
#
```

Verificación del daño

El pc atacante y victima tienen la misma MAC

```
^C
--- 7.41.1.20 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2043ms
rtt min/avg/max/mdev = 0.416/0.471/0.532/0.047 ms
root@Docker:/home/admin# arp -n
Address          HWtype  HWaddress           Flags Mask           Iface
7.41.1.50         ether    50:00:01:9b:00:02    C                    eth1
10.177.0.1         ether    02:42:c6:2d:86:2a    C                    eth0
7.41.1.20         ether    50:00:01:99:00:01    C                    eth1
root@Docker:/home/admin# arp -n
Address          HWtype  HWaddress           Flags Mask           Iface
7.41.1.50         ether    50:00:01:9b:00:02    C                    eth1
10.177.0.1         ether    02:42:c6:2d:86:2a    C                    eth0
7.41.1.20         ether    50:00:01:9b:00:02    C                    eth1
root@Docker:/home/admin# arp -n
Address          HWtype  HWaddress           Flags Mask           Iface
7.41.1.50         ether    50:00:01:9b:00:02    C                    eth1
10.177.0.1         ether    02:42:c6:2d:86:2a    C                    eth0
7.41.1.20         ether    50:00:01:9b:00:02    C                    eth1
```

Contramedida

Habilitar DAI en el switch y Port Security para limitar MAC por puerto.

```
Switch#
Switch>
Switch>en
Switch#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Switch(config)#interface range e0/0 - 2
Switch(config-if-range)# switchport mode access
Switch(config-if-range)# switchport port-security
Switch(config-if-range)# switchport port-security maximum 1
Switch(config-if-range)# switchport port-security violation shutdown
Switch(config-if-range)#ip arp inspection vlan 1
Switch(config)#interface e1/0
Switch(config-if)# ip arp inspection trust
Switch(config-if)#
```

DHCP Spoofing — Servidor DHCP Falso (Rogue DHCP)

Protocolo: DHCP (RFC 2131)

Objetivo del Laboratorio

Demostrar cómo un atacante puede desplegar un servidor DHCP fraudulento que responda a los clientes antes que el servidor legítimo, asignando parámetros de red maliciosos para realizar un ataque MitM automático.

Objetivo del Script

Implementar un servidor DHCP falso que responda al handshake DORA antes que el servidor legítimo (Router R1), asignando como gateway la IP de Kali y un DNS controlado por el atacante.

Parámetros del Script

Parámetro	Descripción	Default
-i / --interface	Interfaz de red	eth2
--pool	Rango de IP a asignar	7.41.10.200-220
--gateway	Gateway a anunciar (IP de Kali)	7.41.10.50
--dns	Servidor DNS malicioso	IP del atacante
--netmask	Máscara de subred	255.255.255.0
--lease	Tiempo de arrendamiento (s)	600

Requisitos para Utilizar la Herramienta

- Python 3 con Scapy: apt install python3-scapy -y
- Privilegios root (sudo)
- IP forwarding habilitado: echo 1 > /proc/sys/net/ipv4/ip_forward
- Ser más rápido que el servidor DHCP legítimo (R1) al responder Discover
- Kali en el mismo segmento L2 que las víctimas y el servidor DHCP legítimo

Funcionamiento del Script

El script escucha paquetes DHCP Discover en broadcast (puertos 67/68 UDP).

Al recibir un Discover, construye y envía un DHCP Offer con parámetros maliciosos:

gateway = 7.41.10.50 (Kali) | dns = servidor del atacante | IP del pool falso

Cuando el cliente envía DHCP Request, el servidor falso confirma con DHCP ACK.

El cliente completa el handshake DORA y configura su red con los parámetros del atacante.

Ejecución del Ataque

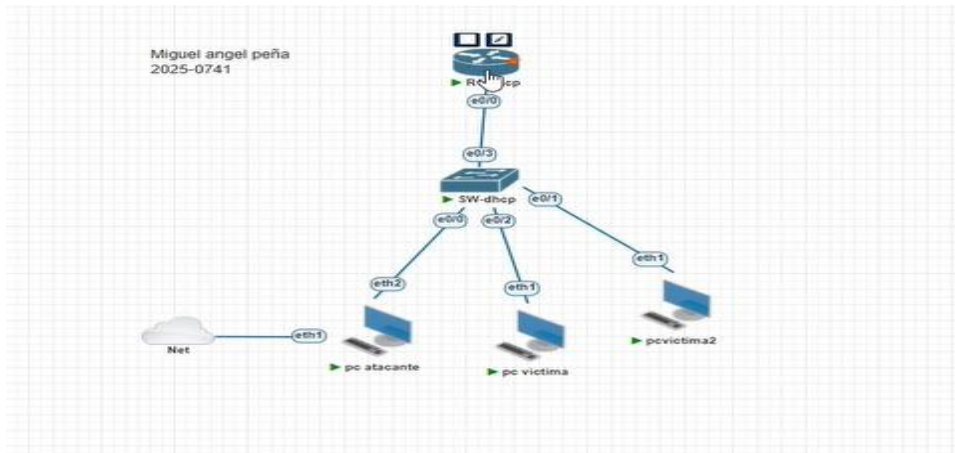
```
!!!      DHCP SPOOFING (ROGUE SERVER) - LABORATORIO SEGURIDAD      !!!
!!!      !!SOLO PARA USO EDUCATIVO Y AUTORIZADO!!      !!!
!!!
[+] Servidor DHCP Rogue activo:
    Interfaz : eth2
    Nuestra IP: 7.41.10.50
    Gateway anunciado : 7.41.10.50
    DNS anunciado : 8.8.8.8
    Pool disponible : 21 IPs
[!] Escuchando solicitudes DHCP... (Ctrl+C para detener)
[>>>] DISCOVER de 50:00:01:9f:00:01
[>>>] OFFER enviado a 50:00:01:9f:00:01 IP: 7.41.10.200 | GW: 7.41.10.50 | DNS: 8.8.8.8
[>>>] REQUEST de 50:00:01:9f:00:01
[>>>] ACK enviado 50:00:01:9f:00:01 recibí7.41.10.200
[>>>] DISCOVER de 50:00:01:a2:00:01
[>>>] OFFER enviado a 50:00:01:a2:00:01 IP: 7.41.10.201 | GW: 7.41.10.50 | DNS: 8.8.8.8
[>>>] REQUEST de 50:00:01:a2:00:01
[>>>] ACK enviado 50:00:01:a2:00:01 recibí7.41.10.201
[>>>] REQUEST de 50:00:01:9f:00:01
[>>>] ACK enviado 50:00:01:9f:00:01 recibí7.41.10.200
[>>>] REQUEST de 50:00:01:a2:00:01
[>>>] ACK enviado 50:00:01:a2:00:01 recibí7.41.10.201
```

Verificación del Ataque

```
Listening on LPF/eth1/50:00:01:a2:00:01
Sending on LPF/eth1/50:00:01:a2:00:01
Sending on Socket/fallback
DHCPDISCOVER on eth1 to 255.255.255.255 port 67 interval 3 (xid=0xcd143c5e)
DHCPOFFER of 7.41.10.200 from 7.41.10.50
DHCPREQUEST for 7.41.10.200 on eth1 to 255.255.255.255 port 67 (xid=0x5e3c14cd)
DHCPACK of 7.41.10.200 from 7.41.10.50 (xid=0xcd143c5e)
mv: cannot move '/etc/resolv.conf.dhclient-new.932' to '/etc/resolv.conf': Device or re
System has not been booted with systemd as init system (PID 1). Can't operate.
Failed to connect to bus: Host is down
bound to 7.41.10.200 -- renewal in 252 seconds.
admin@pc_victima:~$ ip route show default
default via 7.41.10.50 dev eth1
admin@pc_victima:~$
```


Topologia

Nodos: Router R1 (DHCP legítimo, gateway), switch implícito, Kali (eth2=192.168.10.50), PC-Víctima. Red: 192.168.10.0/24.



Contramedida

DHCP Snooping marca los puertos como trusted (hacia el servidor legítimo) o untrusted (resto). Los puertos untrusted descartan DHCP Offers y ACKs, bloqueando servidores falsos.

```
Switch>en
Switch#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Switch(config)#ip dhcp snooping
Switch(config)#ip dhcp snooping vlan 1
Switch(config)#no ip dhcp snooping information option
Switch(config)#
*Jun  3 03:36:19.199: %CDP-4-DUPLEX_MISMATCH: duplex mismatch discovered on Ethernet0/3 (not half duplex), with Router Ethernet0/0 (half duplex)
Switch(config)#interface e0/3
Switch(config-if)# ip dhcp snooping trust
Switch(config-if)#
Switch(config-if)#
*Jun  3 03:37:16.491: %CDP-4-DUPLEX_MISMATCH: duplex mismatch discovered on Ethernet0/3 (not half duplex), with Router Ethernet0/0 (half duplex)
Switch(config-if)#
```

DHCP Starvation — Agotamiento del Pool DHCP

Objetivo del Laboratorio

Demostrar cómo un atacante puede agotar el pool de direcciones IP del servidor DHCP enviando solicitudes masivas con MAC falsas, impidiendo que clientes legítimos obtengan configuración de red.

Objetivo del Script

Agotar el pool de direcciones IP del servidor DHCP legítimo enviando solicitudes DHCP con cientos de MAC falsas y aleatorias, impidiendo que clientes legítimos obtengan configuración de red (DoS).

Parámetros del Script

Parámetro	Descripción	Default
-i / --interface	Interfaz de red	eth2
-c / --count	Número de solicitudes DHCP	254
--confirm	Completar handshake DORA	Desactivado
--delay	Retardo entre solicitudes (s)	0.05
--timeout	Timeout esperando Offer (s)	2

Requisitos para Utilizar la Herramienta

- Python 3 con Scapy: apt install python3-scapy -y
- Privilegios root (sudo)
- Pool DHCP activo en R1 (7.41.10.101 – 7.41.10.254 = 154 IP)
- Kali en el mismo segmento L2 que el servidor DHCP

Funcionamiento del Script

El script genera MAC aleatorias con OUIs de fabricantes comunes (VMware, VirtualBox, QEMU).

Para cada MAC construye un DHCP Discover con campos xid y hostname completamente aleatorios.

Modo rapido: solo envía Discovers (reserva temporal en la tabla del servidor).

Modo --confirm: espera el Offer y responde con un Request para completar el handshake DORA,

bloqueando cada IP del pool de forma definitiva hasta que expire el lease.

El pool del servidor (7.41.10.101-254 = 154 IP) queda agotado en segundos.

Ejecución del Ataque

```
admin@pc atacante:~$ sudo python3 dhcp_starvation.py -i eth2 -c 254
sudo: unable to resolve host pc atacante: Name or service not known
No protocol specified
No protocol specified

DHCP STARVATION ATTACK - LABORATORIO DE SEGURIDAD
SOLO PARA USO EDUCATIVO Y AUTORIZADO!

[*] Iniciando DHCP Starvation en eth2
[*] Solicitudes: 254 | Confirmar: False | Delay: 0.05s

[-] 1/254 Discovers enviados | 21.2 pkt/s
[-] 21/254 Discovers enviados | 9.6 pkt/s
[-] 41/254 Discovers enviados | 9.6 pkt/s
[-] 61/254 Discovers enviados | 9.4 pkt/s
[-] 81/254 Discovers enviados | 9.5 pkt/s
[-] 101/254 Discovers enviados | 9.5 pkt/s
[-] 121/254 Discovers enviados | 9.5 pkt/s
[-] 141/254 Discovers enviados | 9.5 pkt/s
[-] 161/254 Discovers enviados | 9.5 pkt/s
[-] 181/254 Discovers enviados | 9.5 pkt/s
[-] 201/254 Discovers enviados | 9.5 pkt/s
[-] 221/254 Discovers enviados | 9.5 pkt/s
```

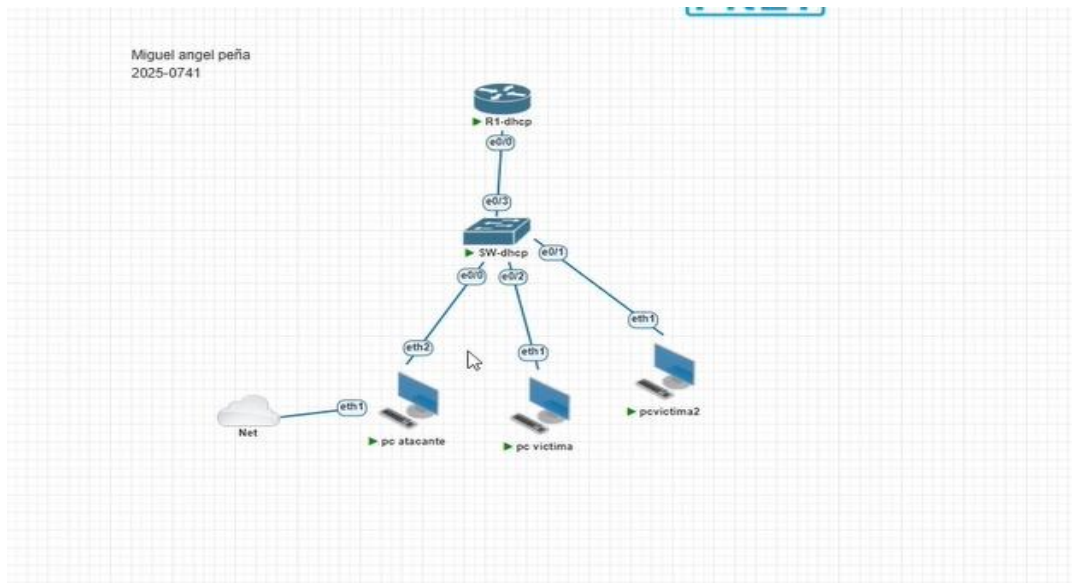

Verificación del Ataque

```
admin@pc victima:~$ sudo dhclient -v eth1
sudo: unable to resolve host pc victima: Name or service not known
Internet Systems Consortium DHCP Client 4.4.1
Copyright 2004-2018 Internet Systems Consortium.
All rights reserved.
For info, please visit https://www.isc.org/software/dhcp/

Listening on LPF/eth1/50:00:01:a5:00:01
Sending on   LPF/eth1/50:00:01:a5:00:01
Sending on   Socket/fallback
DHCPDISCOVER on eth1 to 255.255.255.255 port 67 interval 3 (xid=0x1cfb8213)
```

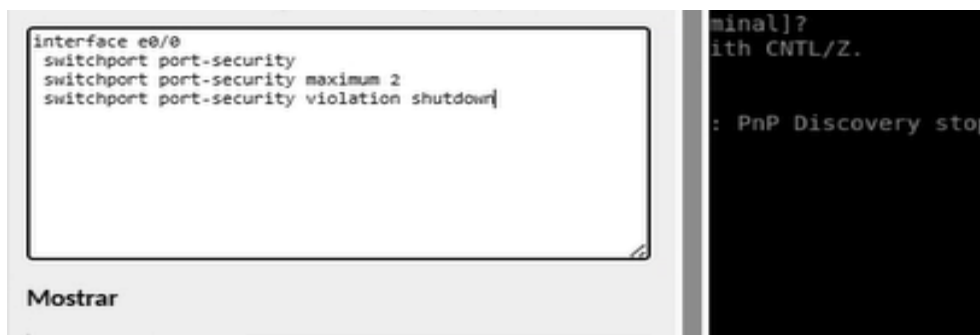
Topología

Topología idéntica al Ataque 03. Nodos: R1 (pool .101-.254) + Kali (atacante) + PC-Víctima.
Red: 7.41.10.0/24.



Contramedida

Limitar el número de MAC por puerto del switch evita que un solo puerto origine cientos de solicitudes con MAC diferentes.



MAC Flooding — Desbordamiento de la Tabla CAM del Switch

Objetivo del Laboratorio

Demostrar cómo el desbordamiento de la tabla CAM (Content Addressable Memory) de un switch Cisco provoca que el dispositivo deje de funcionar como switch inteligente y pase a comportarse como un hub, reenviando todas las tramas a todos los puertos.

Objetivo del Script

Desbordar la tabla CAM del switch con miles de MAC falsas, forzándolo a hacer flooding de frames a todos los puertos y permitiendo la captura del tráfico de toda la red.

Parámetros del Script

Parámetro	Descripción	Ejemplo / Default
-i / --interface	Interfaz de red de salida	eth2
-c / --count	Número de tramas a enviar	50000 (default)
--continuous	Modo continuo hasta Ctrl+C (mantiene tabla desbordada)	(flag)
--threads	Hilos paralelos para mayor velocidad (máx. 4)	1 (default)
--delay	Delay en microsegundos entre tramas	0 (default)
--vlan	ID de VLAN para añadir tag 802.1Q	(opcional)

Requisitos para Utilizar la Herramienta

- Python 3 con Scapy: apt install python3-scapy -y
- Privilegios root (sudo)
- Kali conectada a un puerto del switch objetivo
- Switch Cisco IOSvL2 sin Port Security configurado en el puerto de Kali

Funcionamiento del Script

El script genera tramas Ethernet con MAC de origen y destino completamente aleatorias, opcionalmente con tag 802.1Q. Las tramas se envían en lotes de 100 mediante múltiples hilos paralelos para maximizar la tasa de envío. La tabla CAM del switch (típicamente 8192 entradas) se llena rápidamente y el switch comienza a hacer flooding unicast desconocido a todos los puertos, exponiendo el tráfico de la red completa.

Ejecución del Ataque

```
root@pc atacante:/home/admin# sudo python3 mac_flooding.py -i eth2 --continuous --threads 4
sudo: unable to resolve host pc atacante: Name or service not known
No protocol specified
No protocol specified

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
MAC FLOODING ATTACK - LABORATORIO DE SEGURIDAD          %%%
%%              SOLO PARA USO EDUCATIVO Y AUTORIZADO!    %%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

[*] Iniciando MAC Flooding en eth2
[*] Modo: Continuo | Hilos: 4 | VLAN: Sin tag

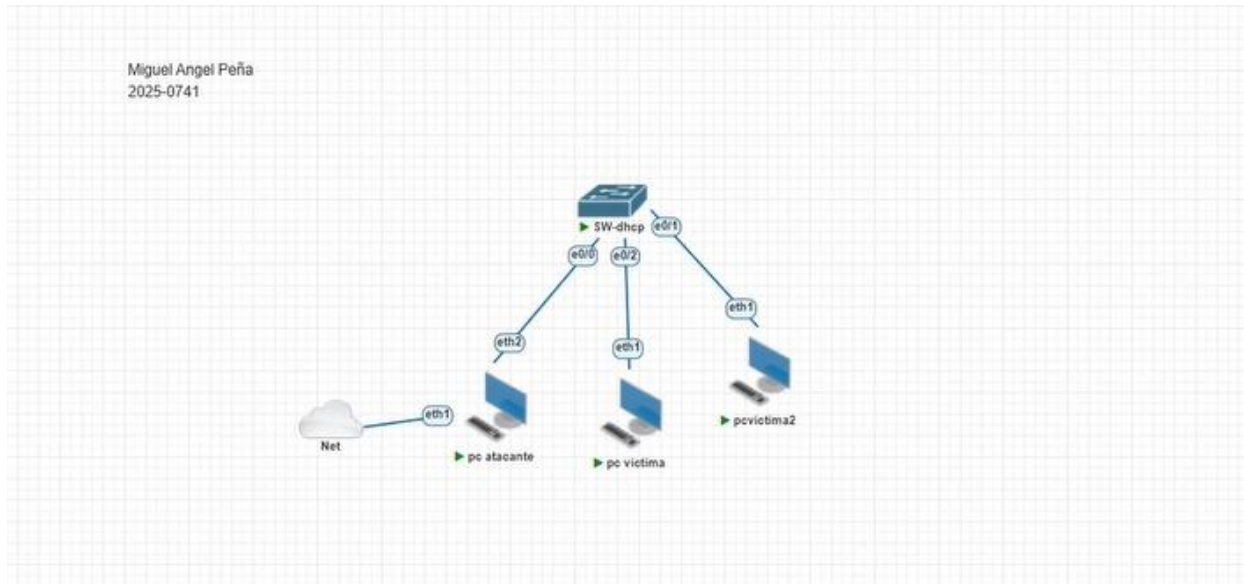
[-] Tramas enviadas: 0 | Tasa: 0 frames/s | Tiempo: 0.1s[!] Modo continuo activo. Presiona Ctrl+C pa
ra detener.
[-] Tramas enviadas: 6,300 | Tasa: 2,031 frames/s | Tiempo: 3.1s
```

Verificación del Ataque

```
Switch#
Switch#
Switch#
Switch#show mac address-table dynamic
      Mac Address Table
-----
Vlan    Mac Address      Type      Ports
-----
1       0003.38ae.d370   DYNAMIC   Et0/0
1       0005.5b58.e41f   DYNAMIC   Et0/0
1       0006.2226.607a   DYNAMIC   Et0/0
1       0007.aed1.617a   DYNAMIC   Et0/0
1       0007.c653.a360   DYNAMIC   Et0/0
1       0008.d30c.d14d   DYNAMIC   Et0/0
1       0011.e74b.93ef   DYNAMIC   Et0/0
1       0012.4f8d.99f1   DYNAMIC   Et0/0
1       0013.9991.ae16   DYNAMIC   Et0/0
1       0014.e108.7045   DYNAMIC   Et0/0
1       001c.54e2.e089   DYNAMIC   Et0/0
1       001e.b984.e3f0   DYNAMIC   Et0/0
1       0023.cdf9.f372   DYNAMIC   Et0/0
1       0024.ac52.d8d2   DYNAMIC   Et0/0
1       0027.0cfa.64e1   DYNAMIC   Et0/0
1       0028.f5a9.d54a   DYNAMIC   Et0/0
1       0029.f961.e6db   DYNAMIC   Et0/0
1       002c.59d0.ba5d   DYNAMIC   Et0/0
1       002e.27fe.4445   DYNAMIC   Et0/0
1       002c.3e43.1a1a   DYNAMIC   Et0/0
```

Topologia

Nodos: Switch Cisco IOSvL2 + Kali (eth2=7.41.10.50) + PC-Víctima. No se requiere router.
Objetivo: tabla CAM (8192 entradas).



Contramedida

cambios en el texto de abajo afectaran al portapapeles remoto.

```
interface e0/0
 switchport mode access
 switchport port-security
 switchport port-security maximum 2
 switchport port-security mac-address sticky
 switchport port-security violation shutdown
```

Verificación de la contramedida:

Port Security con sticky aprende dinámicamente las MAC legítimas y las fija en la configuración, bloqueando cualquier MAC adicional.

```
Mac Entries for Vlan 1:
-----
Dynamic Address Count : 0
Static Address Count : 0
Total Mac Addresses : 0

Total Mac Address Space Available: 221571720

Switch#show mac address-table dynamic
Mac Address Table
-----
Vlan    Mac Address      Type      Ports
-----
Switch#
*Jun  3 04:08:45.425: %PM-4-ERR_DISABLE: psecure-violation error detected on Et0/0, putting Et0/0 in err-disable state
Switch#
*Jun  3 04:08:45.425: %PORT_SECURITY-2-PSECURE VIOLATION: Security violation occurred, caused by MAC address 4016.301e.c9f7 on port Ethernet0/0.
*Jun  3 04:08:46.432: %LINEPROTO-5-UPDOWN: Line protocol on Interface Ethernet0/0, changed state to down
Switch#
*Jun  3 04:08:47.432: %LINK-3-UPDOWN: Interface Ethernet0/0, changed state to down
Switch#show mac address-table dynamic
Mac Address Table
-----
Vlan    Mac Address      Type      Ports
-----
Switch#show mac address-table count

Mac Entries for Vlan 1:
-----
Dynamic Address Count : 0
Static Address Count : 0
Total Mac Addresses : 0

Total Mac Address Space Available: 221571720

Switch#show mac address-table count

Mac Entries for Vlan 1:
-----
Dynamic Address Count : 0
Static Address Count : 0
Total Mac Addresses : 0

Total Mac Address Space Available: 221571720

Switch#
```

STP Root Claim — Usurpación del Root Bridge en Spanning Tree

Objetivo del Laboratorio

Demostrar cómo un atacante puede usurpar el rol de Root Bridge en una topología STP, forzando a todos los switches a reconvergir y causando una interrupción del tráfico de 30 a 50 segundos.

Objetivo del Script

Usurpar el rol de Root Bridge en la topología STP enviando BPDUs de configuración con Bridge Priority 0 (máxima prioridad posible), forzando a todos los switches a reconvergir con Kali como nuevo Root Bridge y causando una interrupción del tráfico de 30 a 50 segundos.

Parámetros del Script

Parámetro	Descripción	Ejemplo / Default
-i / --interface	Interfaz de red de salida	eth2
--priority	Bridge Priority a anunciar	0 (maximo, default)
--hello	Intervalo entre BPDU en segundos	2 (default)
--vlan	ID de VLAN para PVST+ Cisco	10 (ej.)
--mac	MAC del Bridge ID	MAC de la interfaz
--duration	Duración en segundos (0 = continuo)	0 (default)

Requisitos para Utilizar la Herramienta

- Python 3 con Scapy: apt install python3-scapy -y
- Privilegios root (sudo)
- STP/PVST habilitado en los switches (configuración por defecto en Cisco IOS)
- Kali conectada a un puerto trunk o access con acceso a la VLAN objetivo
- Topología con al menos 2-3 switches para observar la reconvergencia

Funcionamiento del Script

El script construye BPDU de configuración STP/PVST+ con Bridge Priority 0 y Root Path Cost 0 (afirmando ser el Root). Los BPDU se envían a la dirección multicast 01:80:C2:00:00:00 cada 2 segundos (hello timer estándar). Los switches reciben los BPDU y, al ver una prioridad menor a la propia ($4096 > 0$), inician el proceso de reelección, reconvergiendo con Kali como Root Bridge y causando un DoS de 30–50 segundos durante la reconvergencia.

Ejecución del Ataque

```
link/ether 50:00:01:ad:00:02 brd ff:ff:ff:ff:ff:ff link-metnsid 0
root@pc atacante:/home/admin# sudo python3 stp_attack2.py -i eth2 --vlan 10
sudo: unable to resolve host pc atacante: Name or service not known
No protocol specified
No protocol specified
[+] Interfaz: eth2 | MAC: 50:00:01:ad:00:02 | Prioridad: 0 | VLAN: 10
[!] Enviando BPDUs cada 2s...

[000] BPDU #0001 enviado | Tiempo: 0s
[000] BPDU #0002 enviado | Tiempo: 2s
[000] BPDU #0003 enviado | Tiempo: 4s
[000] BPDU #0004 enviado | Tiempo: 6s
[000] BPDU #0005 enviado | Tiempo: 8s
[000] BPDU #0006 enviado | Tiempo: 10s
[000] BPDU #0007 enviado | Tiempo: 12s
[000] BPDU #0008 enviado | Tiempo: 14s
[000] BPDU #0009 enviado | Tiempo: 16s
[000] BPDU #0010 enviado | Tiempo: 19s
[000] BPDU #0011 enviado | Tiempo: 21s
[000] BPDU #0012 enviado | Tiempo: 23s
[000] BPDU #0013 enviado | Tiempo: 25s
[000] BPDU #0014 enviado | Tiempo: 27s
[000] BPDU #0015 enviado | Tiempo: 29s
[000] BPDU #0016 enviado | Tiempo: 31s
[000] BPDU #0017 enviado | Tiempo: 33s
[000] BPDU #0018 enviado | Tiempo: 35s
```


Verificación del Ataque

```
VLAN0010
Spanning tree enabled protocol ieee
Root ID    Priority    4106
           Address    aabb.cc00.0100
           This bridge is the root
           Hello Time  2 sec  Max Age 20 sec  Forward Delay 15 sec

Bridge ID   Priority    4106 (priority 4096 sys-id-ext 10)
           Address    aabb.cc00.0100
           Hello Time  2 sec  Max Age 20 sec  Forward Delay 15 sec
           Aging Time  300 sec

Interface    Role Sts Cost      Prio.Nbr Type
-----
Et0/0        Desg FWD 100      128.1    P2p
Et0/1        Desg FWD 100      128.2    P2p

Switch#show spanning-tree vlan 10

VLAN0010
Spanning tree enabled protocol ieee
Root ID    Priority    10
           Address    5000.01ad.0002
           Cost        200
           Port        1 (Ethernet0/0)
           Hello Time  2 sec  Max Age 20 sec  Forward Delay 15 sec

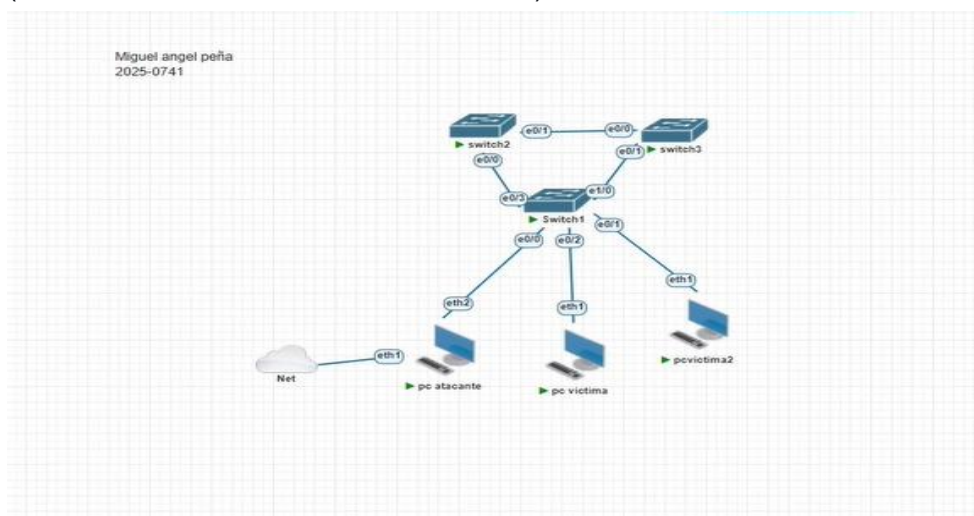
Bridge ID   Priority    4106 (priority 4096 sys-id-ext 10)
           Address    aabb.cc00.0100
           Hello Time  2 sec  Max Age 20 sec  Forward Delay 15 sec
           Aging Time  300 sec

Interface    Role Sts Cost      Prio.Nbr Type
-----
Et0/0        Root FWD 100      128.1    P2p
Et0/1        Desg FWD 100      128.2    P2p

Switch#
```

Topologia

Nodos: SW1 (Root Bridge legítimo, priority 4096) + SW2 + SW3 (VTP Client) + Kali (eth2=7.41.10.50 conectada a SW1 e0/0). VTP domain: EMPRESA.



Contramedida

BPDU Guard coloca el puerto en err-disabled al recibir cualquier BPDU. Root Guard permite recibir BPDU pero bloquea el puerto si el BPDU tuviera mejor prioridad que el Root actual.

```
Switch#
*Jun  3 04:25:27.670: %SYS-5-CONFIG_I: Configured from console by console
Switch#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
Switch(config)#interface e0/0
Switch(config-if)#
Switch(config-if)# spanning-tree guard root
Switch(config-if)# spanning-tree portfast
%Warning: portfast should only be enabled on ports connected to a single
host. Connecting hubs, concentrators, switches, bridges, etc... to this
interface when portfast is enabled, can cause temporary bridging loops.
Use with CAUTION

%Portfast has been configured on Ethernet0/0 but will only
have effect when the interface is in a non-trunking mode.
Switch(config-if)# spanning-tree bpduguard enable
Switch(config-if)#
```

Tabla Resumen — Ataques y Contramedidas

#	Ataque	Protocolo	Impacto Principal	Contramedida Cisco IOS
01	CDP DoS	CDP	Alta CPU · tabla CDP agotada	no cdp run / no cdp enable
02	ARP MitM	ARP (RFC 826)	Intercepción total del tráfico	ip arp inspection vlan + DAI
03	DHCP Spoofing	DHCP (RFC 2131)	MitM automático + DNS malicioso	ip dhcp snooping + trust uplink
04	DHCP Starvation	DHCP (RFC 2131)	DoS: clientes sin IP	port-security maximum 2
05	MAC Flooding	Ethernet 802.3	Switch en modo hub · sniffing	port-security + mac sticky
06	STP Root Claim	STP 802.1D/PVST+	DoS 30–50 s · control topología	bpduguard + spanning-tree guard root